

debug.h

A Battle-Hardened Debugging Header for C23/C++23

API Reference & User Guide

GitHub Copilot
EdgeOfAssembly/RetroCodeMess

December 2025
Version 2.0

Contents

1	Introduction	3
1.1	Key Features	3
1.2	Quick Start	3
2	Configuration	3
2.1	Configuration Macros	3
2.2	Log Levels	4
3	Logging Macros	4
3.1	Basic Logging	4
4	Function Tracing	4
4.1	Entry and Exit	4
5	Variable Inspection	5
5.1	DEBUG_VAR	5
5.2	DEBUG_PTR	5
6	Memory Allocation	6
7	Assertions	6
7.1	DEBUG_ASSERT	6
7.2	DEBUG_CHECK	6
8	Backtrace	6
9	Loop Tracing	7
10	Output Redirection	7
10.1	File Output	7
10.2	Custom Handler	7
11	Thread Safety	7

12 GDB Integration	8
12.1 DEBUG_BREAK	8
12.2 Debugging Tips	8
13 Compilation	8
13.1 Debug Build	8
13.2 Release Build	8
13.3 Thread-Safe Build	8
14 Platform Support	8
15 Complete Example	8
16 API Reference Summary	10
17 License	10

1 Introduction

debug.h is a feature-complete, header-only debugging library for C23 and C++23 that provides powerful debugging capabilities with zero runtime cost in release builds.

1.1 Key Features

- **Five Log Levels:** TRACE, DEBUG, INFO, WARN, ERROR
- **Function Tracing:** Entry/exit logging with return values
- **Type-Safe Inspection:** Uses C11 `_Generic` for automatic formatting
- **Cross-Platform:** Works on Linux, macOS, and Windows
- **Thread-Safe:** Optional mutex locking
- **Timestamps:** Optional date/time prefixes
- **ANSI Colors:** Configurable color output
- **Output Redirection:** File output and custom handlers
- **Zero Overhead:** Compiles out completely with `-DNDEBUG`

1.2 Quick Start

```

1 #define DEBUG_LEVEL 5           // Full verbosity
2 #define DEBUG_TIMESTAMPS      // Add timestamps
3 #define DEBUG_COLORS          // Color output
4 #include "debug.h"
5
6 int main(void) {
7     DEBUG_INFO("Hello, debugging world!");
8
9     int x = 42;
10    DEBUG_VAR(x); // Output: VAR x = 42
11
12    return 0;
13 }
```

2 Configuration

2.1 Configuration Macros

Define these **before** including debug.h:

Macro	Default	Description
DEBUG_LEVEL	5	Verbosity level (0–5)
DEBUG_TIMESTAMPS	undefined	Enable timestamp prefixes
DEBUG_COLORS	undefined	Enable ANSI color output
DEBUG_THREAD_SAFE	undefined	Enable mutex locking
NDEBUG	undefined	Disable all debug code

Table 1: Configuration macros

2.2 Log Levels

Level	Constant	Description	Color
1	LOG_ERROR	Errors only	Red (bold)
2	LOG_WARN	Warnings and above	Yellow
3	LOG_INFO	Info and above	Green
4	LOG_DEBUG	Debug and above	Cyan
5	LOG_TRACE	Everything (verbose)	Magenta

Table 2: Log levels and colors

3 Logging Macros

3.1 Basic Logging

Logging Macros

```
DEBUG_TRACE(fmt, ...); // Most verbose
DEBUG_DEBUG(fmt, ...);
DEBUG_INFO(fmt, ...);
DEBUG_WARN(fmt, ...);
DEBUG_ERROR(fmt, ...); // Least verbose
DEBUG_LOG(level, fmt, ...); // Explicit level
```

Example:

```
1 DEBUG_INFO("Processing item %d of %d", current, total);
2 DEBUG_WARN("Cache miss for key: %s", key);
3 DEBUG_ERROR("Failed to open file: %s (errno=%d)", filename, errno);
```

Output:

```
[INFO ] main.c:10 process_items: Processing item 5 of 100
[WARN ] cache.c:42 lookup: Cache miss for key: user_123
[ERROR] file.c:15 open_file: Failed to open file: data.txt (errno=2)
```

4 Function Tracing

4.1 Entry and Exit

Function Tracing Macros

```
DEBUG_ENTER(fmt, ...); // Log function entry
DEBUG_EXIT(ret);        // Log exit and return value
```

Example:

```
1 int factorial(int n) {
2     DEBUG_ENTER("(n=%d)", n);
3     DEBUG_ASSERT(n >= 0, "Negative input!");
4
5     if (n <= 1) {
```

```

6         DEBUG_EXIT(1);
7     }
8
9     int result = n * factorial(n - 1);
10    DEBUG_VAR(result);
11    DEBUG_EXIT(result);
12 }

```

Output:

```

[TRACE] math.c:2 factorial: ENTER (n=5)
[TRACE] math.c:2 factorial: ENTER (n=4)
[TRACE] math.c:2 factorial: ENTER (n=3)
[TRACE] math.c:2 factorial: ENTER (n=2)
[TRACE] math.c:2 factorial: ENTER (n=1)
[TRACE] math.c:6 factorial: EXIT (return: 1)
[TRACE] math.c:10 factorial: VAR result = 2
[TRACE] math.c:11 factorial: EXIT (return: 2)
...

```

5 Variable Inspection

5.1 DEBUG_VAR

The `DEBUG_VAR` macro uses C11 `_Generic` to automatically format variables based on their type.

Supported Types

- `int`, unsigned `int`
- `long`, unsigned `long`
- `double`
- `char*`
- `void*`

Example:

```

1 int count = 42;
2 double pi = 3.14159;
3 char *name = "Alice";
4 void *ptr = malloc(100);
5
6 DEBUG_VAR(count);    // VAR count = 42
7 DEBUG_VAR(pi);       // VAR pi = 3.141590
8 DEBUG_VAR(name);     // VAR name = Alice
9 DEBUG_VAR(ptr);      // VAR ptr = 0x7f...

```

5.2 DEBUG_PTR

For pointers with size descriptions:

```

1 char *buffer = malloc(1024);
2 DEBUG_PTR(buffer, "1024 bytes");
3 // Output: PTR buffer @ 0x7f... (size: 1024 bytes)

```

6 Memory Allocation

Memory Macros

```
void* ptr = DEBUG_MALLOC(size);
DEBUG_FREE(ptr);
```

Example:

```
1 typedef struct {
2     int id;
3     char name[32];
4 } User;
5
6 User *user = (User*)DEBUG_MALLOC(sizeof(User));
7 // Output: MALLOC 0x55... (size: 40 bytes)
8
9 // ... use user ...
10
11 DEBUG_FREE(user);
12 // Output: FREE 0x55...
```

7 Assertions

7.1 DEBUG_ASSERT

Hard assertion that aborts on failure with backtrace:

```
1 void *ptr = malloc(1024);
2 DEBUG_ASSERT(ptr != NULL, "Allocation failed for %zu bytes", 1024);
```

7.2 DEBUG_CHECK

Soft check that logs warning but continues:

```
1 DEBUG_CHECK(value >= 0, "Warning: negative value %d", value);
2 // Program continues even if check fails
```

8 Backtrace

```
1 DEBUG_BACKTRACE();
```

Output (Linux):

```
[ERROR] main.c:42 risky_function: BACKTRACE (5 frames):
[ERROR] #0: ./prog(risky_function+0x37) [0x55...]
[ERROR] #1: ./prog(main+0x15c) [0x55...]
[ERROR] #2: /lib/x86_64-linux-gnu/libc.so.6(+0x2a1ca) [0x7f...]
[ERROR] #3: /lib/x86_64-linux-gnu/libc.so.6(__libc_start_main+0x8b)
[ERROR] #4: ./prog(_start+0x25) [0x55...]
```

Note: Compile with `-rdynamic` for function names.

9 Loop Tracing

```
1 DEBUG_LOOP_START("Processing %d items", count);
2 for (int i = 0; i < count; i++) {
3     DEBUG_LOOP_ITER(i, count);
4     process(items[i]);
5 }
6 DEBUG_LOOP_END("Completed");
```

Output:

```
[TRACE] process.c:10 process_items: LOOP START Processing 100 items
[DEBUG] process.c:12 process_items: LOOP ITER 0/100
[DEBUG] process.c:12 process_items: LOOP ITER 1/100
...
[TRACE] process.c:15 process_items: LOOP END Completed
```

10 Output Redirection

10.1 File Output

```
1 FILE *logfile = fopen("debug.log", "w");
2 DEBUG_SET_OUTPUT(logfile);
3
4 DEBUG_INFO("This goes to the file");
5 // ... more logging ...
6
7 DEBUG_RESET_OUTPUT(); // Back to stderr
8 fclose(logfile);
```

10.2 Custom Handler

```
1 void my_handler(const char* level, const char* file,
2                 int line, const char* func, const char* msg) {
3     // Custom logging logic (e.g., send to network, database)
4     syslog(LOG_INFO, "[%s] %s:%d %s: %s", level, file, line, func, msg)
5     ;
6 }
7 DEBUG_SET_HANDLER(my_handler);
```

11 Thread Safety

Enable thread-safe logging:

```
1 #define DEBUG_THREAD_SAFE
2 #include "debug.h"
```

Compile with: `-pthread`

Note: Uses `pthread_mutex_t` in C, `std::mutex` in C++.

12 GDB Integration

12.1 DEBUG_BREAK

Insert a breakpoint:

```
1 if (suspicious_condition) {
2     DEBUG_BREAK(); // Triggers SIGTRAP
3 }
```

12.2 Debugging Tips

1. Compile with `-g` for symbols
2. Use `-rdynamic` for backtrace function names
3. In GDB: catch signal `SIGTRAP` to catch `DEBUG_BREAK`
4. Use `bt` in GDB to see call stack

13 Compilation

13.1 Debug Build

```
gcc -std=c23 -g -O0 -Wall -Wextra -rdynamic -o prog prog.c
```

13.2 Release Build

```
gcc -std=c23 -O2 -DNDEBUG -o prog prog.c
```

13.3 Thread-Safe Build

```
gcc -std=c23 -g -O0 -Wall -Wextra -rdynamic -pthread -o prog prog.c
```

14 Platform Support

Platform	Backtrace	Break	Notes
Linux	Yes (execinfo.h)	Yes (<code>__builtin_trap</code>)	Use <code>-rdynamic</code>
macOS	Yes (execinfo.h)	Yes (<code>__builtin_trap</code>)	Full support
Windows (MSVC)	Yes (dbghelp)	Yes (<code>__debugbreak</code>)	Link <code>dbghelp.lib</code>
Windows (MinGW)	Fallback	Yes (<code>__builtin_trap</code>)	Limited backtrace

Table 3: Platform support matrix

15 Complete Example

```
1 /*
2  * example.c - Complete debug.h usage example
3  */
4
5 #define DEBUG_LEVEL 5
6 #define DEBUG_TIMESTAMPS
```



```

7 #define DEBUG_COLORS
8 #include "debug.h"
9 #include <string.h>
10
11 typedef struct {
12     int id;
13     char name[32];
14     double score;
15 } Student;
16
17 Student* create_student(int id, const char* name, double score) {
18     DEBUG_ENTER("(id=%d, name=%s, score=%.2f)", id, name, score);
19
20     Student* s = (Student*)DEBUG_MALLOC(sizeof(Student));
21     DEBUG_ASSERT(s != NULL, "Failed to allocate Student");
22
23     s->id = id;
24     strncpy(s->name, name, sizeof(s->name) - 1);
25     s->name[sizeof(s->name) - 1] = '\0';
26     s->score = score;
27
28     DEBUG_PTR(s, "Student struct");
29     DEBUG_VAR(s->id);
30     DEBUG_VAR(s->score);
31
32     DEBUG_EXIT(s);
33 }
34
35 void process_students(Student** students, int count) {
36     DEBUG_ENTER("(count=%d)", count);
37     DEBUG_CHECK(count > 0, "No students to process");
38
39     double total = 0.0;
40
41     DEBUG_LOOP_START("Calculating average");
42     for (int i = 0; i < count; i++) {
43         DEBUG_LOOP_ITER(i, count);
44         total += students[i]->score;
45     }
46     DEBUG_LOOP_END();
47
48     double average = total / count;
49     DEBUG_VAR(average);
50     DEBUG_INFO("Average score: %.2f", average);
51 }
52
53 int main(void) {
54     Student* s1 = create_student(1, "Alice", 95.5);
55     Student* s2 = create_student(2, "Bob", 87.3);
56
57     Student* students[] = {s1, s2};
58     process_students(students, 2);
59
60     DEBUG_FREE(s1);
61     DEBUG_FREE(s2);
62
63     return 0;
64 }

```

16 API Reference Summary

Macro	Description
<i>Logging</i>	
DEBUG_TRACE/DEBUG/INFO/WARN/ERROR	Log at specified level
DEBUG_LOG(level, fmt, ...)	Log at explicit level
<i>Function Tracing</i>	
DEBUG_ENTER(fmt, ...)	Log function entry
DEBUG_EXIT(ret)	Log exit with return
<i>Inspection</i>	
DEBUG_VAR(var)	Print variable value
DEBUG_PTR(ptr, desc)	Print pointer info
<i>Memory</i>	
DEBUG_MALLOC(size)	Logged malloc
DEBUG_FREE(ptr)	Logged free
<i>Assertions</i>	
DEBUG_ASSERT(expr, ...)	Hard assert (aborts)
DEBUG_CHECK(expr, ...)	Soft check (continues)
DEBUG_BACKTRACE()	Print stack trace
<i>Loops</i>	
DEBUG_LOOP_START/ITER/END	Loop tracing
<i>Output</i>	
DEBUG_SET_OUTPUT(file)	Redirect to file
DEBUG_RESET_OUTPUT()	Reset to stderr
DEBUG_SET_HANDLER(fn)	Custom handler
<i>GDB</i>	
DEBUG_BREAK()	Insert breakpoint
DEBUG_IF(expr)	Conditional block
DEBUG_ABORT(msg, ...)	Abort with message

Table 4: Complete API reference

17 License

MIT License

Copyright (c) 2025 EdgeOfAssembly

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.